

UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO – UFRSA

Centro de Ciências Exatas e Naturais – CCEN

Departamento de Computação - DC

Graduação em Ciência da Computação

Disciplina: Estrutura de Dados II

Prof.: Paulo Henrique Lopes Silva

Prática Offline 2

1. Cache eviction.

- **Cache eviction** é o processo de remoção de itens do cache quando ele atinge sua capacidade máxima, a fim de liberar espaço para novos dados.
- Esse processo é essencial para manter a eficiência do cache, evitando que ele fique cheio e incapaz de armazenar novos dados.

2. Principais políticas de cache eviction.

- LRU (*Least Recently Used*):
 - Remove o item que não foi acessado há mais tempo.
 - Baseia-se na suposição de que os dados acessados recentemente serão acessados novamente em breve.
 - Aplicações: Sistemas operacionais (gestão de páginas de memória), navegadores web (gestão de cache de recursos).
- FIFO (*First In, First Out*):
 - Remove o item mais antigo no cache, ou seja, aquele que foi inserido primeiro.
 - Simples de implementar, mas pode não ser tão eficiente quanto outras políticas em termos de acesso frequente.
 - Aplicações: Sistemas de filas de processamento, algumas implementações de cache de hardware.
- LFU (*Least Frequently Used*):
 - Remove o item que foi acessado com menos frequência.
 - Baseia-se na suposição de que os dados menos utilizados são menos importantes.
 - Aplicações: Caches de dados em bases de dados, alguns tipos de cache em sistemas distribuídos.
- Remoção aleatória:
 - Remove um item aleatório do cache.
 - Simples de implementar e pode ser útil em algumas situações onde a carga de trabalho é imprevisível.
 - Aplicações: Algumas caches de hardware, algoritmos de amostragem.

3. Aplicações:

- **Cache eviction** é aplicada em várias áreas, incluindo:
 - Sistemas operacionais: Gestão de memória virtual, caches de disco, buffers de arquivos.
 - Navegadores web: Cache de recursos como imagens, scripts e páginas web.
 - Bancos de dados: Caches de consultas, caches de dados frequentemente acessados.
 - Sistemas de arquivos: Caches de diretórios, caches de metadados.
 - Redes de distribuição de conteúdo (CDNs): Caches de conteúdo web para reduzir a latência e melhorar a performance.
 - Hardware: Caches de CPU (L1, L2, L3), caches de discos rígidos e SSDs.

4. Operações:

- **Hit**: procurar e achar um dado na cache.
- **Miss**: procurar um dado na base de dados quando este não for encontrado na cache.
- Principal objetivo é criar políticas que maximizem **hits** e minimizem **miss**.

5. Indexação em bancos de dados.

- Indexação é a criação de estruturas auxiliares (índices) que permitem localizar rapidamente registros em uma tabela, sem precisar examinar linha por linha.
- É semelhante ao índice de um livro, onde uma página sobre um tema pode ser localizada rapidamente sem precisar folhear tudo.

6. Um cenário de armazenamento em bancos de dados.

- **Armazenamento físico:**
 - Em sistemas de banco de dados, os registros geralmente são armazenados em blocos ou páginas de disco, não em uma sequência estritamente ordenada.
 - A ordem em que os registros são inseridos não garante uma ordem física dos dados no disco.
 - Além disso, ao longo do tempo, com inserções, deleções e atualizações, o armazenamento pode se tornar fragmentado.
- **Armazenamento lógico (sem índice):**
 - Sem um índice, o banco precisa fazer uma varredura sequencial para encontrar um registro com base em um critério.

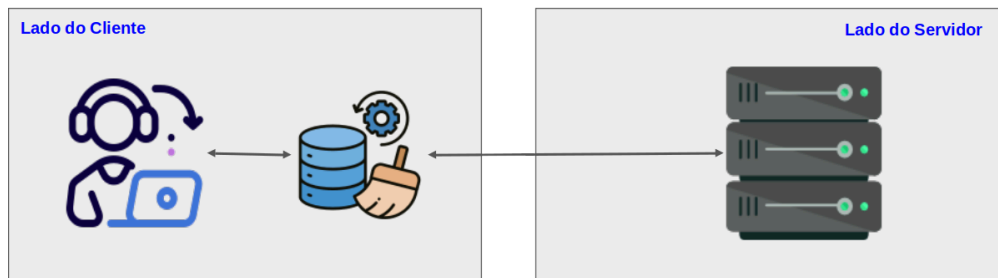
7. [TAREFA] Simulação de Sistema de *Streaming*: Cliente-Servidor.

Esta simulação modela o processo de autenticação e recuperação de conteúdo, focando na eficiência de busca (lado do cliente) e na organização de grandes volumes de dados (lado do servidor).

- **O lado do cliente (*Frontend/App*).**
 - O cliente atua como a interface de consumo.
 - Para evitar requisições desnecessárias à rede (que são custosas), ele utiliza um cache.
 - Estrutura: Árvore AVL.
 - Função: Armazena os títulos acessados recentemente ou metadados de login.
 - Por que usar?
 - Diferente de uma lista comum, a AVL garante que o tempo de busca seja sempre logarítmico, ou seja, $O(\log n)$. Isso assegura que, mesmo com muitos itens no cache, a verificação seja instantânea.
 - *Eviction* (Esvaziamento): Quando o cache atinge um limite, a árvore realiza a remoção de nós para dar lugar a novos dados, simulando a gestão de memória de um dispositivo real (como um celular ou *Smart TV*).
 - Como fazer?
 - É uma decisão de projeto que deve ser bem justificada.
- **O lado do servidor (*Backend/Database*).**
 - O servidor detém o "catálogo mestre".
 - Um filme deve ter, no mínimo, um id, um nome, uma breve sinopse e o ano.
 - Ele precisa lidar com uma quantidade massiva de títulos e simular a persistência em um meio físico.
 - Armazenamento físico: Lista Ligada.
 - Função: Simula a disposição sequencial ou encadeada dos dados em um disco ou memória persistente.
 - Característica: É uma estrutura linear onde cada elemento aponta para o próximo. Por si só, a busca aqui seria lenta ($O(n)$).
 - Indexação: Tabela Hash.
 - Função: Atua como um "mapa de endereços" para a lista ligada.
 - Por que usar?
 - Para não ter que percorrer a lista inteira toda vez que o cliente pede um filme, o servidor usa uma função hash que aponta diretamente para a posição do item na lista (ou para o início de um bloco). Isso reduz o tempo de acesso médio para $O(1)$.
- **Fluxo da simulação.**
 - **Requisição:** O cliente tenta acessar o título "Matrix".
 - Verificação de cache (AVL): O sistema busca na Árvore AVL local.

- *Hit*: O dado é encontrado e exibido imediatamente.
 - *Miss*: O dado não está na AVL. O cliente dispara uma requisição ao servidor.
- Processamento no servidor (Hash + Lista):
 - O servidor recebe o nome do título, calcula o hash para encontrar o índice e acessa o nó correspondente na lista ligada.
 - O valor que cada entrada da tabela hash deve guardar é uma referência para aquele objeto na lista ligada.
- **Resposta e atualização:**
 - O servidor envia o título ao cliente.
 - O cliente exibe o vídeo e insere esse novo título na sua Árvore AVL.
 - Se a AVL estiver cheia, o algoritmo de esvaziamento remove um nó antigo para manter o balanceamento e o limite de memória.
- **Exemplo: O fluxo com IDs (melhor cenário).**
 - Login: O servidor envia o catálogo simplificado: uma lista de pares [ID, Nome].
 - Busca: O usuário clica em "Matrix". O app sabe (ou deve saber) que "Matrix" é o ID 505.
 - Cache (AVL): O sistema busca por 505 na AVL.
 - Sendo um inteiro, as rotações e comparações da AVL são extremamente rápidas.
 - Servidor: Se não estiver no cache, o cliente pede ao servidor: GET /filme/505.
 - Hash: O servidor faz hash(505), cai direto no link (referência) do objeto na lista ligada e devolve os dados.
- **Resultados esperados:**
 - Simular corretamente os mecanismos de cache e indexação:
 - Cache com árvore AVL
 - Lista ligada (dados reais).
 - Indexação de filmes usando uma tabela hash.
- **Execução da simulação:**
 - Inicia o programa:
 - 1000 registros devem ser adicionados na base de dados do servidor.
 - O cache deve ter espaço para guardar 50 registros. Então, 50 registros devem ser armazenados no cache quando o programa iniciar (simular consultas prévias).
 - Após isso, mostrar a interface do cliente.
 - Sugestão: para o console não ficar muito cheio, o cliente pode buscar títulos por categoria (crie as categorias).
 - Realizar:
 - Crie e execute um método para realizar 20 consultas. Dentre elas:
 - Duas inválidas.
 - Seis consultas a registros que estão no cache. Conte e mostre o número de comparações realizadas na requisição.
 - Seis consultas devem ser feitas sem a indexação. Conte e mostre o número de comparações realizadas na requisição.
 - Seis consultas devem ser feitas com a indexação. Conte e mostre o número de comparações realizadas na requisição.
 - Faça uma breve análise sobre os resultados.

8. Representação Gráfica da Simulação.



9. Avaliação.

- O trabalho vale 50% da nota da 2ª unidade.
- Deve ser desenvolvido na linguagem Java usando orientação a objetos.
 - Data de entrega (única): 17/05/2026, somente via sigaa.
 - Formato de envio: seu-nome-pratica-off-2.zip.
- **O projeto é individual.**
 - Em caso de alguma dúvida, o(a) aluno(a) poderá ser chamado para esclarecimentos.
 - As estruturas de dados de cache e indexação devem ser implementadas do zero.
- **Em caso de comprovação de cópia ou fraude, os trabalhos envolvidos receberão a nota ZERO.**

10. Roteiro de apresentação.

- Executar o programa resultante de cada enunciado, questão ou projeto.
- Para cada programa, execute uma operação por vez.
 - Durante a testagem de cada funcionalidade comente sobre os possíveis erros e exceções que podem ocorrer mediante entradas incorretas. Comente também sobre quais erros e exceções seu programa captura.
 - Cite as ferramentas e tecnologias utilizadas.
 - Seu programa implementa todas as funcionalidades requeridas?
- Após a demonstração de funcionamento, é hora de abrir o código. Explique:
 - Como o seu projeto está organizado? (se quiser, use uma representação gráfica para responder).
 - Quais as principais dificuldades e lições aprendidas com o desenvolvimento do projeto?
- Responda as perguntas detalhando o código que você implementou.
- Instruções para gravação:
 - O vídeo deve ter no máximo oito (8) minutos.
 - A gravação será o instrumento avaliativo.
 - Pontos de avaliação:
 - Sequência lógica e domínio do conteúdo (introdução, divisão organizada da apresentação, concatenação de ideias e conclusão).
 - Precisão na comunicação (colocação e entonação da voz, ritmo, dicção e linguagem).
 - Capacidade de argumentação.
 - Domínio e uso de material (organização no uso de recursos de apresentação).
- Sugestões de software para gravação:
 - Google Meet, Zoom, SimpleScreenRecorder, OBS, etc.
- Sobre o envio do vídeo:
 - O projeto deve ser armazenado e disponibilizado por meio de um link.
 - Exemplos: Google Drive, YouTube, Dropbox, etc.